

DOCUMENTAÇÃO DO PROTÓTIPO: MONITORAMENTO DE PC

SUMÁRIO

1 PROBLEMA	2
2 OBJETIVO	2
3 JUSTIFICATIVA	2
4 DESCRIÇÃO DO PROJETO	2
5 FUNCIONALIDADES	2
6 RESULTADOS ESPERADOS	2
7 VISÃO GERAL	3
8 ARQUITETURA E MÓDULOS PRINCIPAIS	3
8.1 Aplicação Web e API (app.py)	3
8.2 Coleta de Dados do Sistema (system_monitor.py)	3

1 PROBLEMA

Atualmente, muitos usuários não possuem ferramentas simples e acessíveis para acompanhar o desempenho do computador. Isso dificulta a identificação de problemas como uso excessivo de CPU, consumo elevado de memória RAM e sobrecarga do sistema, podendo causar lentidão, travamentos e perda de produtividade.

2 OBJETIVO

Desenvolver uma aplicação capaz de monitorar os principais recursos do computador, como CPU, memória RAM e disco, apresentando informações claras ao usuário e gerando alertas em situações críticas.

3 JUSTIFICATIVA

A criação deste sistema é importante para auxiliar na manutenção preventiva e na otimização do desempenho dos computadores. Além disso, o projeto contribui para o desenvolvimento de habilidades práticas em programação, análise de dados e sistemas computacionais.

4 DESCRIÇÃO DO PROJETO

O sistema será desenvolvido em Python, utilizando bibliotecas especializadas em monitoramento, como psutil. A aplicação poderá apresentar uma interface simples, exibindo dados atualizados em tempo real, como porcentagem de uso da CPU, memória e armazenamento.

5 FUNCIONALIDADES

- Monitoramento de CPU em tempo real
- Monitoramento de memória RAM
- Monitoramento de disco
- Exibição de informações atualizadas continuamente
- Geração de alertas em caso de uso elevado

6 RESULTADOS ESPERADOS

Espera-se que o sistema seja leve, eficiente e de fácil utilização, permitindo ao usuário identificar problemas rapidamente e melhorar o desempenho do computador.

7 VISÃO GERAL

Este prototipo eh uma aplicacao web baseada em Flask projetada para monitorar em tempo real os recursos do sistema operacional. Ele coleta metricas de hardware, lista processos ativos (permitindo o encerramento deles), registra o historico em um banco de dados e possibilita a exportacao de relatorios em PDF.

8 ARQUITETURA E MÓDULOS PRINCIPAIS

8.1 Aplicação Web e API (app.py)

O arquivo principal gerencia o servidor Flask, as rotas da API e integra os modulos de banco de dados e agendamento de tarefas.

Principais Rotas:

- **/**: Rota raiz, carrega a interface de usuario (index.html).
- **/api/metrics**: Fornece o estado atual do sistema (CPU, RAM, Disco, Rede, Temperatura) e os ultimos alertas gerados.
- **/api/processes**: Retorna uma lista dos 15 processos com maior consumo de memoria RAM.
- **/api/kill/<pid>**: Rota (metodo POST) para encerrar um processo em execucao usando o seu identificador (PID).
- **/api/history**: Retorna o historico de metricas armazenadas nos ultimos 5 minutos, ideal para plotagem de graficos na interface.
- **/export/pdf**: Gera e envia para download um relatorio PDF com o retrato instantaneo das metricas atuais usando ReportLab.

8.2 Coleta de Dados do Sistema (system_monitor.py)

Este modulo utiliza a biblioteca **psutil** para interagir diretamente com o sistema operacional e hardware.

Funcionalidades de Destaque:

- **Metricas de Hardware:** Mede o uso percentual e absoluto de CPU e RAM, espaco no Disco raiz e temperatura dos sensores da placa.
- **Metricas de Rede:** Calcula a taxa de trafego de rede (Download e Upload em MB/s) verificando a diferenca de bytes transferidos em curtos intervalos de tempo. Tambem captura o IP local e numero de conexoes ativas.
- **Gestao de Processos:** A funcao *get_top_processes* varre os processos em execucao, tratando excecoes para processos encerrados, e retorna os que mais consomem memoria. A funcao *kill_process* atua para encerrar tarefas indesejadas pelo PID.